

Program graph and formal foundations

Methodology: Program Graphs and Intermediate Representations

Program graph

Let $G = (V, E)$ denote a directed graph whose vertices V represent program entities and whose edges E represent relationships. COGANT assigns each node a **kind** (for example function, variable, type) and a **semantic role** (for example definition versus use). Each node and edge may carry:

- ▶ A **stable identifier** for persistence across runs when hashing allows.
- ▶ **Type strings** and attributes recovered from the front end.
- ▶ A **confidence** score in $[0, 1]$ and **provenance** metadata (source code, type system, control flow, heuristic, external tool).

This follows the same philosophical line as code property graphs that fuse AST, control flow, and data flow into one analyzable structure [yamaguchi2014modeling], but orients the representation toward **Generalized Notation Notation (GNN) export** and, optionally, tensorized views of the same graph: node kinds and roles map to discrete indices in both forms, and optional

Formal definitions

This subsection makes the objects manipulated by the pipeline mathematically explicit. The definitions are stated for the shipped v0.5.x engine; where the implementation differs from the general form (for example because an edge kind is not yet emitted by the Python front end), the difference is noted with a forward reference to Section 6.

Definition 1 (Program graph). A **program graph** is a tuple $G = (V, E, \lambda_V, \lambda_E, \tau)$ where

- ▶ V is a finite set of program nodes (modules, classes, methods, functions, and — on languages where the front end emits them — variables and control-flow sites);
- ▶ $E \subseteq V \times V \times K$ is a finite set of typed directed edges drawn from the edge-kind alphabet $K \supseteq \{\text{READS, WRITES, CALLS, CONTAINS, INHERITS, IMPORTS, OBS}\}$;
- ▶ $\lambda_V : V \rightarrow \mathcal{N}$ labels each node with a node kind $\mathcal{N} \supseteq \{\text{MODULE, CLASS, METHOD, FUNCTION, CONFIGURATION, ...}\}$;
- ▶ $\lambda_E : E \rightarrow K$ is the (trivial) projection onto the edge kind;